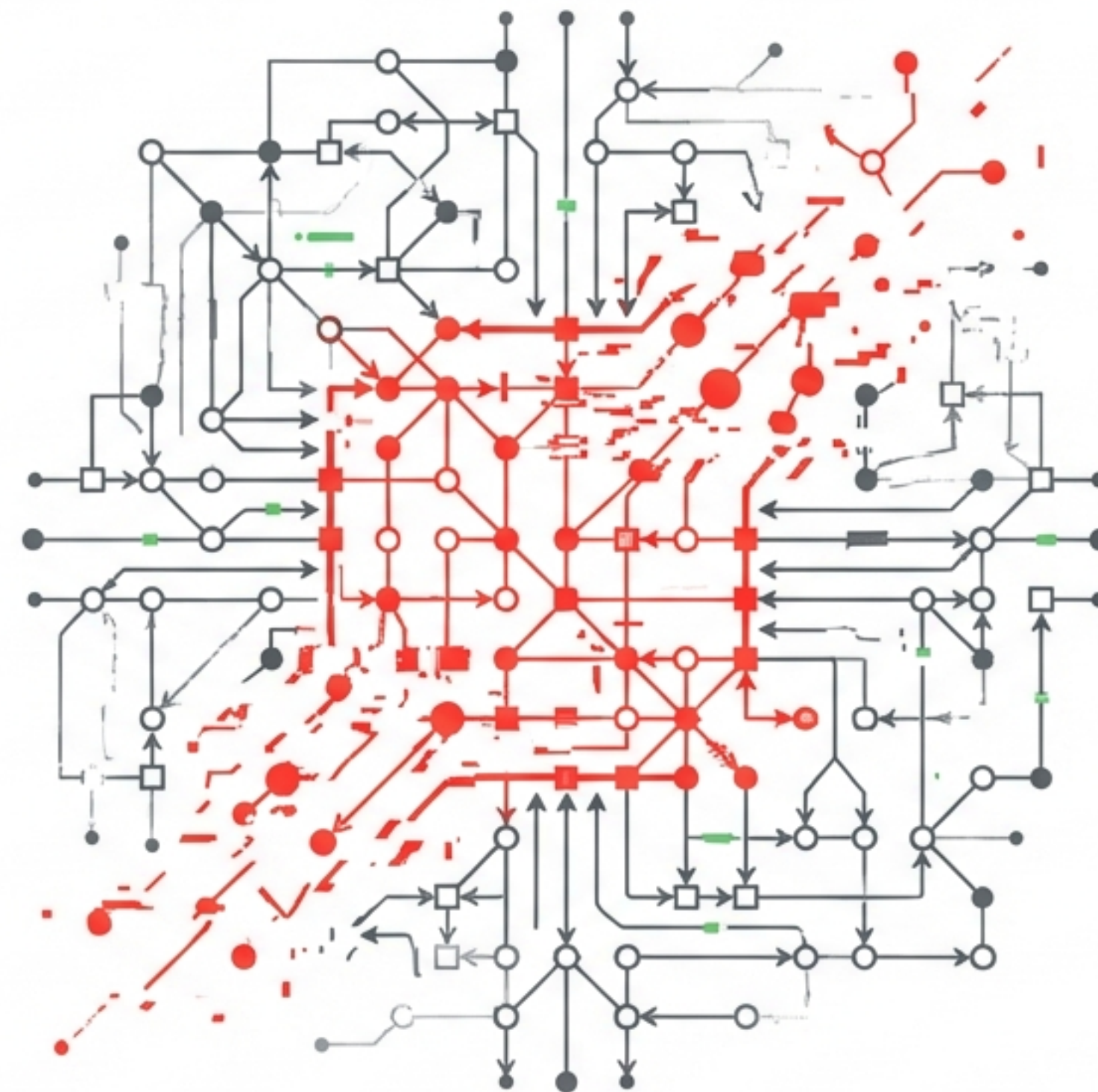


GitHub Platform Lock-In & Risk Assessment

Hidden Dependencies, Data Loss Scenarios,
and the Agent Threat Surface.



The Illusion of Safety: Git is Not GitHub

Core Assumption vs. Operational Reality

The Assumption



We use Git, therefore our history is safe.

✓ **PARTIALLY TRUE** (Code Only)

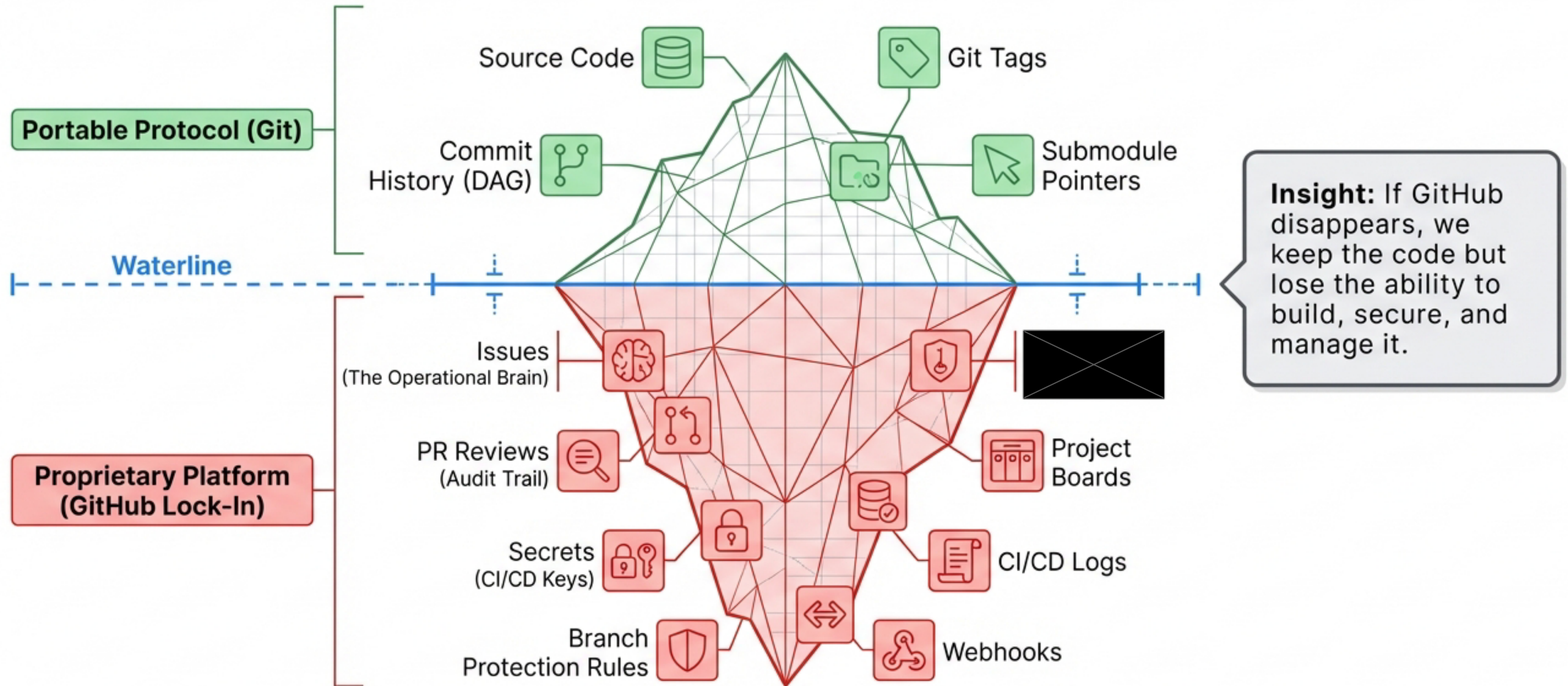
The Risk Reality

 Operational Paralysis	Issues, Sprints, and Logic exist only in proprietary DB. Loss = No instruction set.
 Security Gaps	Branch protection & IAM are "Shadow Config". Recovery defaults to "Unsecured".
 Compliance Failure	PR Reviews & Logs are not in Git. SOC 2 evidence is lost.

Urgency Signals

Recent Instability: Feb 2 & Feb 9, 2026 Outages | **New Threat:** AI Agents (PromptPwnd)

The Visibility Gap: What We Own vs. What We Rent



The Operational Brain: Why Issues Are Critical Infrastructure

The screenshot shows a GitHub issue page for 'Fix Payment Race Condition #402'. The issue is labeled 'Open', 'Bug', and 'P0'. It was created by 'JetBrains Mono' and has a 'GT Green' status. The issue description states: 'Root cause identified in asynchronous transaction processing. Requires locking mechanism. See attached stack traces...'. Below the description is a code block with the following content:

```
See attached stack traces... {
  poolts accesss = eaccesscoladnection {
  }

  // Identified the asynchronous processing.
  const cendig = new ifle.cynitnmoustransactionprocessne();

  if (beethescode != next3) {
    conetflean = itexhecsthactionnactseel();
    return se065f1;
  }

  // Conceptual transaction processing.
  if (erethiesoce, reheatcsi [
    SyrecaBefecclon, tuensactleoTrasterv();
```

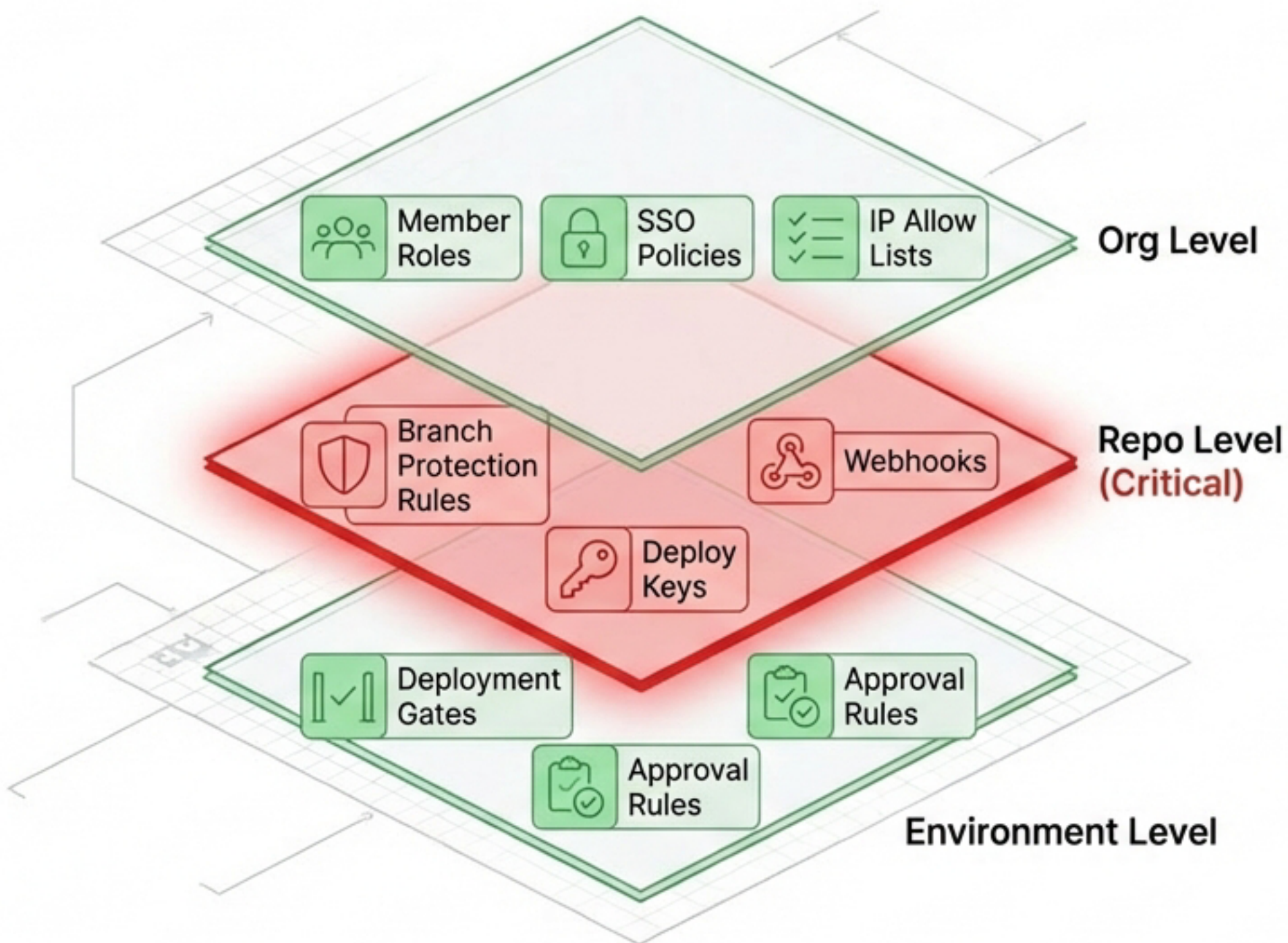
On the right side of the issue, the 'Assignees' section lists 'jdoe' and 'asmith'. The 'Labels' section shows 'No topet'. The 'Incages' section shows 'No labels'. The 'Notification' section shows 'No assignearound'. A large red watermark '404: DATA NOT FOUND' is overlaid on the right side of the screenshot.

The Day 1 Loss Impact:

1. **Loss of Prioritisation:** What is the P0 bug? Backlog vanishes.
2. **Loss of Institutional Knowledge:** Root cause analysis and triage steps are gone.
3. **Loss of Relationships:** Commit messages referencing #402 point to nothing.

Invisible Infrastructure: The Shadow Configuration

Critical settings that exist only as state in the UI.



The Risk: “Recovered” repositories default to “Unsecured”.

-  Main branch becomes writable by all.
-  CI checks are bypassed.
-  History can be overwritten.

The Kill Switch: Secrets & Compliance Evidence

Secrets are Write-Only



Values cannot be read back via API. Loss results in immediate halt of all CI/CD pipelines.

The Compliance Gap



PR Reviews provide the 'Why' behind the code. Missing reviews = Audit Failure for SOC 2 / ISO 27001.

System Fragility: The Warning Signs

February 2026

**6-Hour Actions Outage
(Azure Storage Error)**

Feb 2, 2026



Copilot



CodeQL



Pages

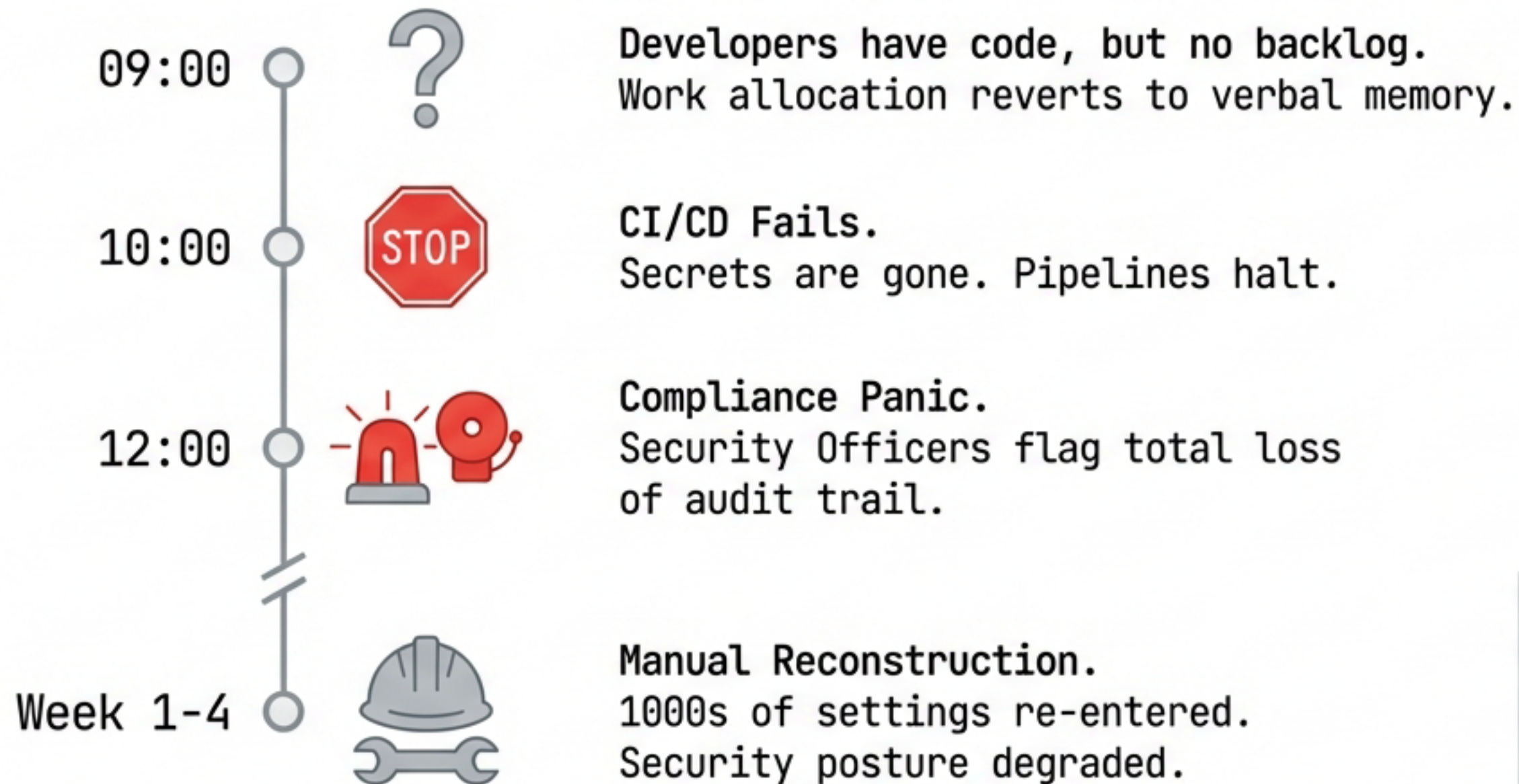
5+ Hour Web UI Outage

Feb 9, 2026

2024 Context: 26 Major Incidents | ~130 Hours Cumulative Disruption

Scenario Analysis: The 'Day 1' Paralysis

Timeline of a Permanent Loss Event (Ransomware/Destructive Attack)



Estimated Recovery:

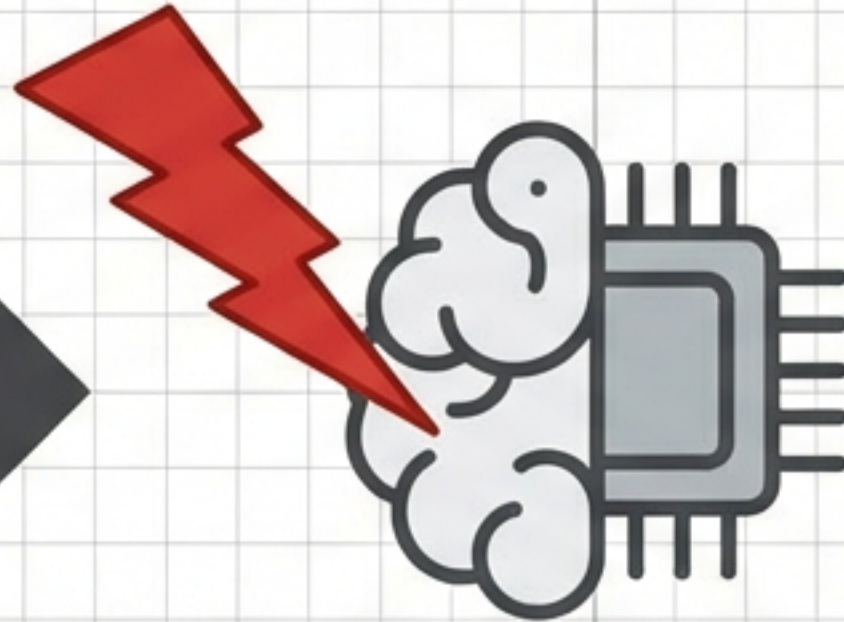
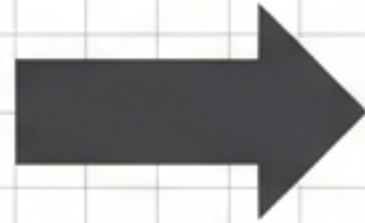
2-4 Weeks to functional state.
History permanently lost.

The Accelerant: AI Agent Threat Surface

The 'PromptPwnd' Vulnerability Class



Malicious
Issue Body



Prompt Injection



Exfiltrate Secrets
/ Delete Data

Mechanism: Agent interprets text data as executable instruction.
Write access enables high-velocity destruction.

Why GitHub Defenses Fail Against Agents



No Undo Button

Deletion of Issues/Labels is instant and permanent.



Generous Rate Limits

5,000 req/hr allows mass destruction in minutes.

OFF/READ ON/ADMIN



Coarse Granularity

Permissions are Read vs Admin. No 'Edit but don't Delete'.



Blind Spots

No anomaly detection for mass deletion events.

An agent can do more damage in 5 minutes than a rogue employee in a week.

The Shared Responsibility Model



Your Responsibility (The Gap)

- Data Protection (Issues/Code)
- IAM / Access Control
- Configuration Management
- AI Agent Permissions



GitHub's Responsibility

- Infrastructure Uptime
- Physical Security
- Network Availability



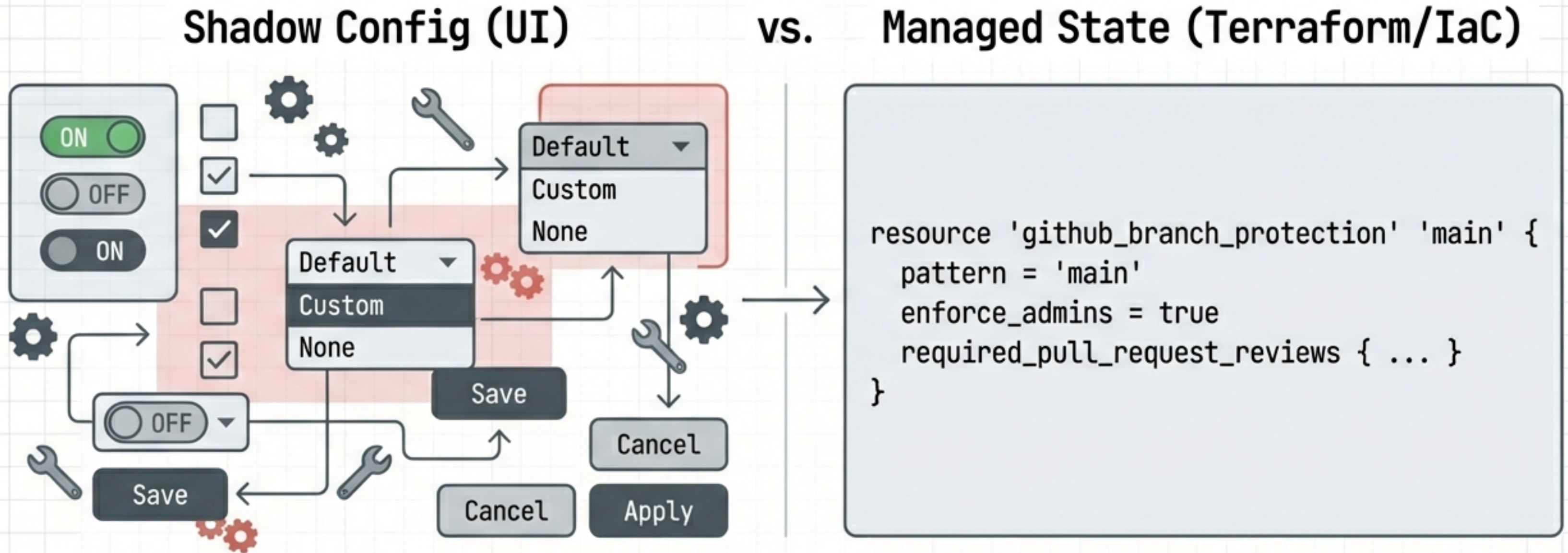
*"GitHub explicitly states data protection is the user's responsibility.
There is no SLA guarantee for data restoration."*

Mitigation Strategy: The 'P0' Priorities

**IMMEDIATE
ACTION REQUIRED**

- ☐ **Automated Issue Backup**
Full export via API (metadata + comments). Test restore process.
- ☐ **Externalise Secrets**
Move to Vault/AWS. GitHub becomes a consumer, not storage.
- ☐ **Back up PR Reviews**
Essential for compliance evidence.

Configuration as Code: The Long-Term Fix



Benefits: ✓ Disaster Recovery, Drift Prevention, Audit Trail.

Securing the AI Layer



No Admin
Tokens

Never grant admin
scope to agents.



Write
Gates

Maintain Write-
Permission Gates
for Codex/Claude.



Input
Sanitation

Treat Issue bodies
as untrusted input.



Human-in-
the-Loop

No 'YOLO mode'
on production.

Conclusion: From Utility to Critical Vendor

Treat GitHub as you would AWS or Azure.

Back it up. Lock it down. Manage it with code.



Next Steps

1. Audit current backup coverage (Issues vs. Code).
2. Initiate Terraform import of Repo Config.
3. Review AI Agent permissions **immediately**.